



The Clocks of C++

Knowing When (and Why) to Use Each One

SANDOR DARGO



20
26



Who Am I?

Sándor DARGÓ

Senior Engineer at **Spotify**

Enthusiastic blogger: sandordargo.com

Curious oenophile

Fortunate father of two



Have you ever had to deal with a
bug caused by the wrong
representation of time?

Why This Talk?

Recently worked on an instrumentation project

Had to deal with timestamps, clock types, durations and more...

Moreover, had to transform different kinds of time points

The best way to learning is by sharing

And if I'm wrong, someone will correct me 🙄

Agenda

The building blocks of `<chrono>`

The main standard clocks

Some more clocks

Conversions

Building our own clocks

Time Zones

The Building Blocks of Chrono

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

The Problems of “C” Time

Lack of type safety

- Easy to mix quantities of different units

Lack of composability

- Hard to build higher level abstractions

<chrono> Is Built Around Three Pillars

Durations - `std::chrono::duration`

Timepoints - `std::chrono::time_point`

Clocks - `std::chrono::*_clock`

`std::chrono::duration` represents intervals

Rep is for tick count

Period is tick length in seconds (ratio)

```
std::ratio<1, 1'000'000'000> == std::nano
```

```
std::ratio<1, 1'000'000> == std::micro
```

```
std::ratio<1, 1'000> == std::milli
```

```
template<
    class Rep,
    class Period =
std::ratio<1>
> class duration;
```

Some common durations

<code>std::duration<long long, std::ratio<1, 1'000'000'000>></code>	<code>std::chrono::nanoseconds</code>
<code>std::duration<long long, std::nano></code>	<code>std::chrono::nanoseconds</code>
<code>std::duration<long long, std::milli></code>	<code>std::chrono::milliseconds</code>
<code>std::duration<long long></code>	<code>std::chrono::seconds</code>

Fractions can be greater than one!

<code>std::chrono::duration<int, std::ratio<60>></code>	<code>std::chrono::minutes</code>
<code>std::chrono::duration<int, std::ratio<3600>></code>	<code>std::chrono::hours</code>
<code>std::chrono::duration<int, std::ratio<86400>> (C++20)</code>	<code>std::chrono::days</code>
<code>std::chrono::duration<int, std::ratio<604800>> (C++20)</code>	<code>std::chrono::weeks</code>
<code>std::chrono::duration<int, std::ratio<2629746>> (C++20)</code>	<code>std::chrono::months</code>
<code>std::chrono::duration<int, std::ratio<31556952>> (C++20)</code>	<code>std::chrono::years</code>

duration lets us safely comparing different units

```
// https://godbolt.org/z/vvee8n4sj
#include <chrono>

int main() {
    using namespace std::chrono_literals;

    constexpr auto d1 = 500ms;    // std::chrono::milliseconds
    constexpr auto d2 = 2s;       // std::chrono::seconds

    static_assert(d1 < d2);

    return 0;
}
```

time_point represents a point in time

```
// https://godbolt.org/z/oEPqxqPGc
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono_literals;
    std::chrono::time_point now =
        std::chrono::system_clock::now();
    std::cout << now << '\n';
}
/*
2026-09-16 17:19:32.959809437
*/
```

Subtract time_points to get a duration

```
// https://godbolt.org/z/b5os1Wcav
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono_literals;
    std::chrono::time_point now = std::chrono::system_clock::now();
    std::chrono::time_point now2 = std::chrono::system_clock::now();
    auto d = now2 - now;
    auto d2 = now - now2;
    std::cout << d << ' ' << d2 << '\n';
    return 0;
}
/*
622ns -622ns
*/
```

Shift a `time_point` by a duration

```
// https://godbolt.org/z/echMG1YKr
#include <chrono>
#include <iostream>

int main() {
    using namespace std::chrono_literals;
    std::chrono::time_point now = std::chrono::system_clock::now();
    std::chrono::time_point earlier = now - 10min;
    std::chrono::time_point later = now + 10min;
    std::cout << earlier << '\n' << now << '\n' << later << '\n';
    return 0;
}
/*
2026-09-16 17:10:31.979537580
2026-09-16 17:20:31.979537580
2026-09-16 17:30:31.979537580
*/
```

`std::chrono::*_clock` - the source of `time_points`

A *clock* needs two things:

A starting point, a.k.a. an epoch

A tick rate

A typical example is `system_clock`

Epoch: *1st of January 1970*

Tick rate: *1s*

What makes a clock, really?

There is a type trait!

`std::chrono::is_clock` requires:

Nested `rep`, `period`, `duration` and `time_point` types

`time_point` is typically

`std::chrono::time_point<Clock>`

Providing a static `now()` returning a `time_point`

Having a constant `is_steady` boolean

There are plenty of clocks!

C++11 introduced three standard clocks (all in `std::chrono`)

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 brought us some more:

`utc_clock`

`tai_clock`

`gps_clock`

`file_clock`

system_clock

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

`system_clock` represents

System-wide wall-clock

Reflects real world calendar time

Typically uses the Unix epoch (though not mandated)

Easy conversion to old-school C `time_t`

```
// https://godbolt.org/z/9WaxjPsjs
#include <chrono>
#include <iostream>

int main() {
    auto now = std::chrono::system_clock::now();
    std::time_t now_c = std::chrono::system_clock::to_time_t(now);
    auto now_tp = std::chrono::system_clock::from_time_t(now_c);
    std::cout << "Current C time: " << std::ctime(&now_c) << '\n';
    std::cout << "Current C++ time: " << now_tp << '\n';
    return 0;
}
/*
Current C time: Mon Jan 12 16:28:43 2026
Current C time: Wed Sep 16 17:21:09 2026
Current C++ time: 2026-09-16 17:21:09.000000000
*/
```

Use it when it has to make sense to humans

In logging messages

In file metadata

For saving / displaying dates in a UI

Don't use when you measure durations

It's not good for elapsed time because the wall-clock time can jump

The user can change the system clock manually

Network Time Protocol can adjust the system time

Encapsulate clocks for testing purposes

Encapsulate them

Use a real clock for production code

And a fake one for tests

```
struct Clock {  
    virtual ~Clock() = default;  
    virtual std::chrono::system_clock::time_point  
        now() const = 0;  
};  
  
struct RealClock : Clock {  
    std::chrono::system_clock::time_point now() const  
        override {  
            return std::chrono::system_clock::now();  
        }  
};  
  
struct FakeClock : Clock {  
    std::chrono::system_clock::time_point provided{};  
    std::chrono::system_clock::time_point now() const  
        override { return provided; }  
};
```

steady_clock

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

It's a monotonic clock

It will never go backwards

Ideal for measuring elapsed time

Default choice for measuring how long something takes

Even the standard library uses it

But it cannot tell you the time

Its epoch is humanly meaningless

It's intentionally unspecified

Therefore you cannot rely on it

It's usually based on

- System boot time

- Monotonic kernel counter

- Or some hardware timers

Use it for measuring durations

Must not serialize and compare across processes or restarts

Cannot convert it to `time_t`

Never assume that `time_since_epoch()` has a meaning

high_resolution_clock

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

You might think that...

It's **the** clock with the highest resolution

It's **the** go to solution if you need accuracy

But accuracy != precision

Moreover...

... it's usually just an alias

The standard says it represents “clocks with the shortest tick period”

It also says, in practice, it “may be a synonym for `system_clock` or `steady_clock`”

You can figure it out

```
int main() {
    using std::chrono::high_resolution_clock;
    std::cout << std::boolalpha
        << "high_resolution_clock is steady? "
        << high_resolution_clock::is_steady << "\n"
        << "high_resolution_clock == steady_clock?"
        << std::is_same_v<high_resolution_clock,
            std::chrono::steady_clock>
        << "\n"
        << "high_resolution_clock == system_clock? "
        << std::is_same_v<high_resolution_clock,
            std::chrono::system_clock>
        << "\n";
}

/*
Possible output:
high_resolution_clock is steady? false
high_resolution_clock == steady_clock? false
high_resolution_clock == system_clock? true
*/
```

So should you use it?

Not if you need portable steadiness and monotonicity

Only use it, if you've confirmed that it gives you benefits on the target platform

Some platforms provide a truly finer-grained steady timer

You can check it with this code:

```
std::chrono::high_resolution_clock::is_steady &&  
!std::is_same_v<std::chrono::high_resolution_clock,  
                std::chrono::steady_clock>
```

C++20 clocks

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

utc_clock: The Leap-Second-Aware Clock

Gives real-world wall-clock time, including leap seconds

Might return non-linear results due leap seconds

`std::chrono::leap_second` can be taken from timezone DB

Helps with logging/tracing when you must align with real-world UTC

`tai_clock`: International Atomic Time (TAI)

Has an earlier epoch (1/1/1958) and has no leap seconds

Runs uniformly

UTC is falling behind (started with 10s diff, now 37s)

Useful for scientific computing, simulations

Or as a reference scale for other custom clocks

gps_clock: Time for Satellites

Its epoch is *6th January 1980*, has no leap seconds

It's drifting from `utc_clock` (currently 18s diff)

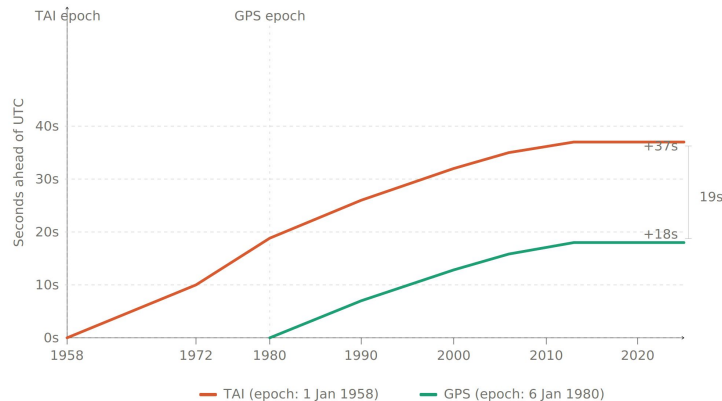
Useful for

Navigation,

Mapping,

Telemetry,

And systems consuming GNSS timestamps



`file_clock`: A Clock for Filesystem Timestamps

The clock underlying `std::filesystem::file_time_type`

Its epoch is implementation-defined satisfying `TrivialClock` requirements

No more lossy and surprising conversion for file timestamps

Well defined conversions to `system_clock` OR to `utc_clock`

Great for interoperability with filesystems

local_t: Civil Time Without a Time Zone

A pseudo-clock to indicate that a `time_point` represents local civil time w/o a TZ

```
auto local =  
    std::chrono::local_time<std::chrono::minutes>{...};
```

It becomes meaningful when paired with a TZ

```
auto tz = std::chrono::locate_zone("Europe/Berlin");  
auto sys_time = tz->to_sys(local);
```

The design forces you handle TZ rules, DST transitions, etc.

Inter-clock conversions

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

Inter-clock comparisons can be tricky

Each clock defines its own epoch

1970-01-01 00:00:00 UTC for `system_clock`

1980-01-06 00:00:00 UTC for `gps_clock`

1958-01-01 00:00:00 TAI for `tai_clock`

Unspecified for `steady_clock`

Etc.

What is the time diff between 1970-01-01 and some arbitrary boot-time counter?

C++20 brought us `std::chrono::clock_cast`

Allows converting a `time_point` from one clock to another

When the conversion is defined

When is a conversion defined?

A conversion is defined if:

The clocks have a known, stable mathematical relationship

A `clock_time_conversion<From, To>` specialization exists

Conversions are available for

Clocks having known epoch and offsets:

`system_clock`

`utc_clock`

`tai_clock`

`gps_clock`

`file_clock`

```
using namespace std::chrono;
```

```
auto utc_now = utc_clock::now();
```

```
auto tai_now =  
    clock_cast<tai_clock>(utc_now);
```

What if there is no well-defined and stable
conversion available?

You can try to manually correlate them

Take two timepoints “at the same time”

Save their offset

Use it later to estimate the other time

```
auto system_now =  
    std::chrono::system_clock::now();  
  
auto steady_now =  
    std::chrono::steady_clock::now();
```

Useful, but the offset might become invalid

NTP sync

Manual clock change

Don't use it for long running processes

```
// The offset lets you map steady  
// to system later on.  
  
auto offset = system_now - steady_now;  
  
auto estimated_system_time =  
    some_steady_tp + offset;
```

Duration casting and precision issues

When casting to coarser units, we can lose precision

```
using namespace std::chrono_literals;

auto ns = 1500ns; // 1500 nanoseconds

auto us =
    std::chrono::duration_cast<
        std::chrono::microseconds>(ns);

// us == 1 microsecond
// (the remaining 500ns are lost)
```

Duration casting and precision issues

When casting to coarser units, we can lose precision

When casting to finer units, we might introduce imaginary precision

```
std::chrono::milliseconds ms{1};

auto ns2 =
    std::chrono::duration_cast<
        std::chrono::nanoseconds>(ms);

// ns2 == 1'000'000ns
// but we didn't *measure*
// at nanosecond precision
```

Be explicit while casting to reveal intentions

C++20 gives us `std::chrono::floor`, `ceil`, and `round`

Even if not technically necessary use them

To make intent clear

To show you're losing information

```
// https://godbolt.org/z/K7GPdKMn9
using namespace std::chrono_literals;
auto original = 1499ns;

// Round to the *nearest* microsecond
auto rounded_us =
    std::chrono::round<std::chrono::microseconds>(
        original);
// rounded_us == 1us

// We've effectively decided that 1499ns ≈ 1μs
// The remaining 499ns are gone by design.
std::cout << original.count() << "ns\n";
std::cout << rounded_us.count() << "us\n";
/*
1499ns
1us
*/
```

How to build your own clock?

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

The right question to ask, why?

Sometimes a real clock is the wrong tool

You need manually controlled time for **testing**

In **game engines, physics simulations** you might want to choose when and how time increases

When **reproducing events**, a replay clock can step through with original timestamps

Reminder: What makes a clock, really?

There is a type trait!

`std::chrono::is_clock` requires:

Nested `rep`, `period`, `duration` and `time_point` types

`time_point` is typically

`std::chrono::time_point<Clock>`

Providing a static `now()` returning a `time_point`

Having a constant `is_steady` boolean

Let's build a manually advanced clock

// <https://godbolt.org/z/Keq1xxv37>

```
class ManualClock {
public:
    using rep = std::int64_t;
    using period = std::nano;
    using duration = std::chrono::duration<rep, period>;
    using time_point = std::chrono::time_point<ManualClock>;

    static time_point now() noexcept { return time_point(current_time); }

    static constexpr bool is_steady = true;

    static void advance(duration d) noexcept { current_time += d; }

private:
    static inline time_point current_time{duration{0}};
};

static_assert(std::chrono::is_clock_v<ManualClock>);
```

And now let's use it

```
// https://godbolt.org/z/Keq1xxv37
// ...

int main() {
    using namespace std::chrono_literals;

    ManualClock::advance(500ms);
    auto t1 = ManualClock::now();

    ManualClock::advance(200ms);
    auto t2 = ManualClock::now();

    std::cout << (t2 - t1).count() << " ns\n";
    return 0;
}
```

Interoperability with Standard Clocks

There is no automatic conversion

If you want meaningful conversions:

Does it follow `system_clock` with some offset?

Does it map to real time at all?

Is its epoch aligned with Unix epoch or something arbitrary?

Another example: A scaling clock

```
// https://godbolt.org/z/639zjsbd8
struct ScaledClock {
    using rep          = std::int64_t;
    using period       = std::nano;
    using duration     = std::chrono::duration<rep, period>;
    using time_point   = std::chrono::time_point<ScaledClock>;

    static time_point now() noexcept {
        auto real = std::chrono::steady_clock::now().time_since_epoch();
        auto real_in_duration = std::chrono::duration_cast<duration>(real);

        // Scale the count as double, then cast back to rep
        double scaled_count = static_cast<double>(real_in_duration.count()) * multiplier;
        return time_point{duration{static_cast<rep>(scaled_count)}};
    }

    static constexpr bool is_steady = false;
    static void set_speed(double mul) { multiplier = mul; }
    static inline double multiplier{1.0};
};
```

Encapsulating clocks for testing can be enough

Just encapsulate them

Use a real clock for production code

And a fake one for tests

Pass them as a dependency

```
struct Clock {  
    virtual ~Clock() = default;  
    virtual std::chrono::system_clock::time_point  
        now() const = 0;  
};  
  
struct RealClock : Clock {  
    std::chrono::system_clock::time_point now() const  
        override {  
            return std::chrono::system_clock::now();  
        }  
};  
  
struct FakeClock : Clock {  
    std::chrono::system_clock::time_point provided{};  
    std::chrono::system_clock::time_point now() const  
        override { return provided; }  
};
```

But why do we need a fake clock for testing?

How do you test the properties of a stored record?

Can't really mock `system_clock`

Approximation is out of question

```
struct Record {  
    int value;  
    std::chrono::system_clock::time_point  
        created_at;  
};  
  
std::vector<Record> db;  
  
void store(int value) {  
    db.emplace_back(  
        value,  
        std::chrono::system_clock::now());  
}
```

We can inject a clock

We made `store` testable

Easy to understand

Fits to what we now about *Dependency Injection*

```
void store(int value, const Clock&
clock) {
    db.emplace_back(value, clock.now());
}

void testFoo() {
    FakeClock clock;
    clock.provided = system_clock::now();
    store(42, clock);
    assert(db[0].created_at ==
           clock.provided);
}
```

We could also benefit from *concepts*

Expressive, zero-overhead

Fits nicely template-heavy code

Provides compile-time guarantees

Requires C++20

```
template <class C>
concept SystemClockLike =
    std::chrono::is_clock_v<C> && requires {
        { C::now() } ->
        std::same_as<std::chrono::system_clock::time_point>;
    };

void store(int value, SystemClockLike auto clock) {
    db.emplace_back(value, clock::now());
}

void testFoo() {
    FakeClock clock;
    clock.provided = system_clock::now();
    store(42, clock);
    assert(db[0].created_at == clock.provided);
}
```

Pass time providers as a dependency

No custom types

No inheritance

No concepts

Arguably the simplest option

```
using TimeProvider = std::function<
    std::chrono::system_clock::time_point()>;

void store(int value, TimeProvider tp) {
    db.emplace_back(value, tp());
}

void testFoo() {
    auto expected =
        std::chrono::system_clock::now()
    TimeProvider tp = [&expected]() {
        return expected;
    };
    store(42, tp);
    assert(db[0].created_at == expected);
}
```

Practical Advice for Designing Custom Clocks

Don't reinvent the wheel

Keep epochs consistent

Consider `is_steady` carefully

C++20 brought TimeZone support

The Building Blocks of Chrono

`system_clock`

`steady_clock`

`high_resolution_clock`

C++20 clocks

Inter-clock conversions

How to build your own clock

C++20 TimeZone support

What does this timestamp mean
to a human, in a specific place, at
a specific moment?

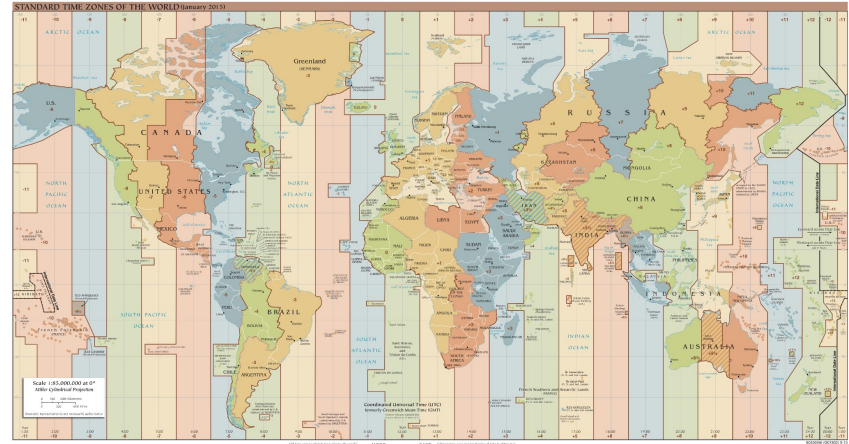
Why TZs matter?

Offsets matter, but it's more than that:

DST rules change

Offsets depend on historical and political decisions

Local Time is not a stable concept



Before C++20 you could...

Ignore the problem and hope that UTCish time is good enough

Use third party libraries

Provide some in-house support

What C++20 Introduced

A full time zone database

Reliable conversions between

System time (`std::chrono::sys_time`)

Local civil time (`std::chrono::local_time`)

Is it about system time or UTC time?

`system_clock` counts seconds since the Unix epoch

Unix time closely tracks UTC

Informally, we often talk about UTC ↔ local time conversion

But it's not coming from `utc_clock`

Converting to Local Time

Get a hold on the timezone db

```
// https://godbolt.org/z/rj3jvPxGh
```

Locate the TZ representation

It might throw!

Pass a `sys_time` to `to_local()`

Can also pass a

`local_time` to `to_sys()`

```
const auto& timezone_db =  
    std::chrono::get_tzdb();  
const auto* riviera_tz =  
    timezone_db.locate_zone("Europe/Paris");  
  
std::chrono::sys_time now =  
    std::chrono::system_clock::now();  
std::chrono::local_time local_now =  
    riviera_tz->to_local(now);  
std::chrono::sys_time reconverted_sys_now =  
    riviera_tz->to_sys(local_now);
```

Zoned Time: Making Time and Place Explicit

Passing around raw time points can be error-prone

`zoned_time` bundles a time zone and a point in time

```
std::chrono::zoned_time meeting{  
    "Europe/Paris",  
    std::chrono::system_clock::now()};
```

Can use `local_time` directly

Just use `local_days` and a time

```
std::chrono::local_time meeting = std::chrono::local_days{2025y/3/30} + 9h;
```

When this local time is associated with a time zone, the library can correctly handle:

- ambiguous times during DST fall-back

- skipped times during spring-forward changes

Time to Conclude

The Big Mental Shift: Time Is Not Just a Number

We have to think about

Which clocks are we using?

Is it a duration or a timepoint?

Are we talking about monotonic or civil time?

Can the value jump?

Do we want to measure, order or just print something readable?

The Key Is Picking The Right Clock

Use Case	Clock
Measure elapsed time	<code>steady_clock</code>
Timeouts, retries, deadlines	<code>steady_clock</code>
Logging human-readable timestamps	<code>system_clock</code>
Wall-clock / civil time logic	<code>system_clock</code> + <code>time_zone</code>
Simulations or testing	custom clocks
Leap-second-aware time scales	<code>utc_clock</code>
Physics, astronomy	<code>tai_clock</code>

Durations Are Your Friend

They are separately modeled from timepoints

- They are strongly typed

- Cheap

- Compose well

- Make units explicit

Design For Testing

You cannot really mock `system_clock::now()`

Wrap it

Build your own testing clock

Or just use an existing one

Treat time as a dependency

Inject your time provider(s)



The Clocks of C++

Knowing When (and Why) to Use Each One

SANDOR DARGO



20
26

